ASD – Software architecture

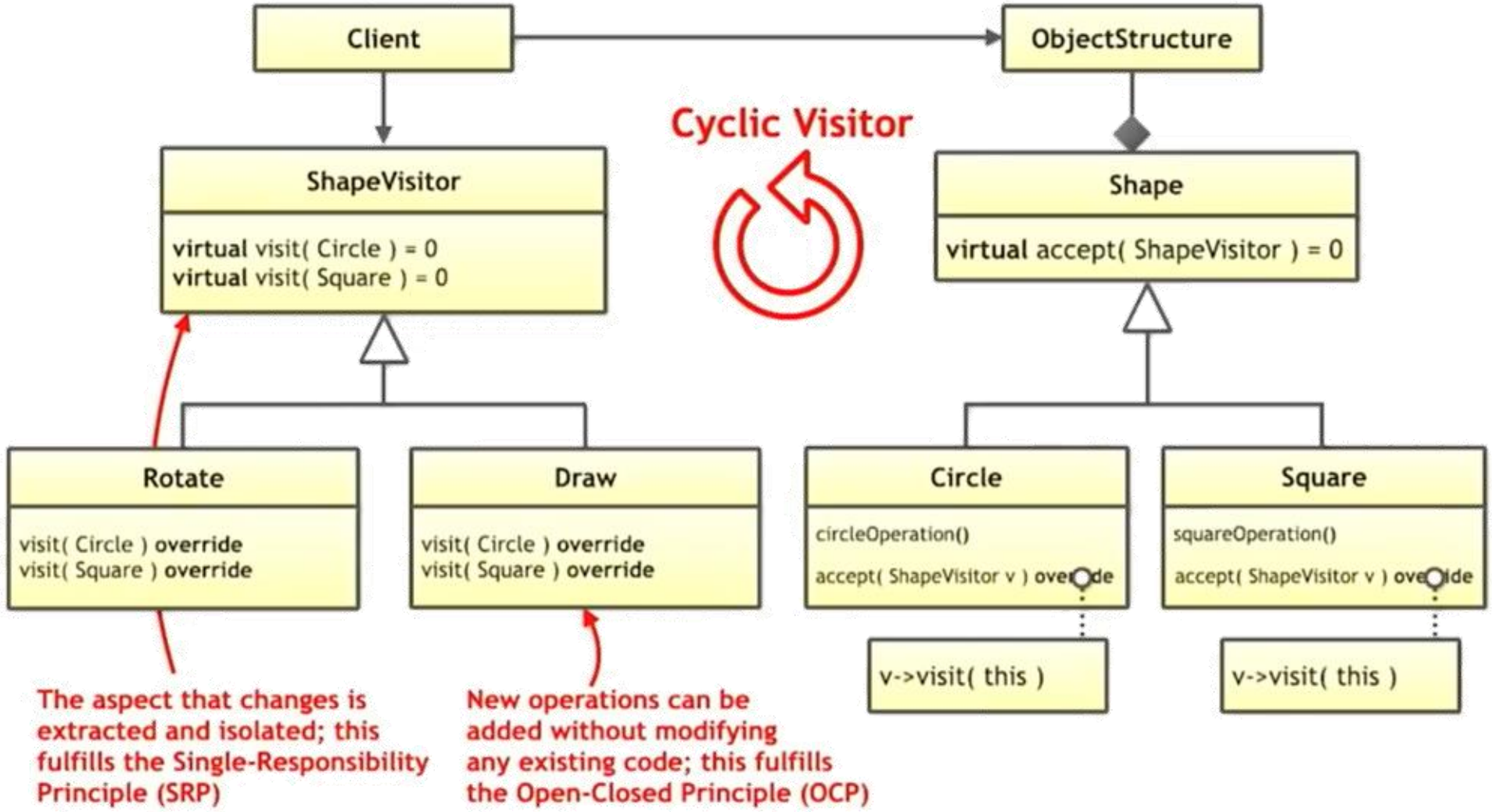
Aspect oriented programming





Why AOP (Aspect oriented programming)?

Remember our visit pattern



Why AOP?

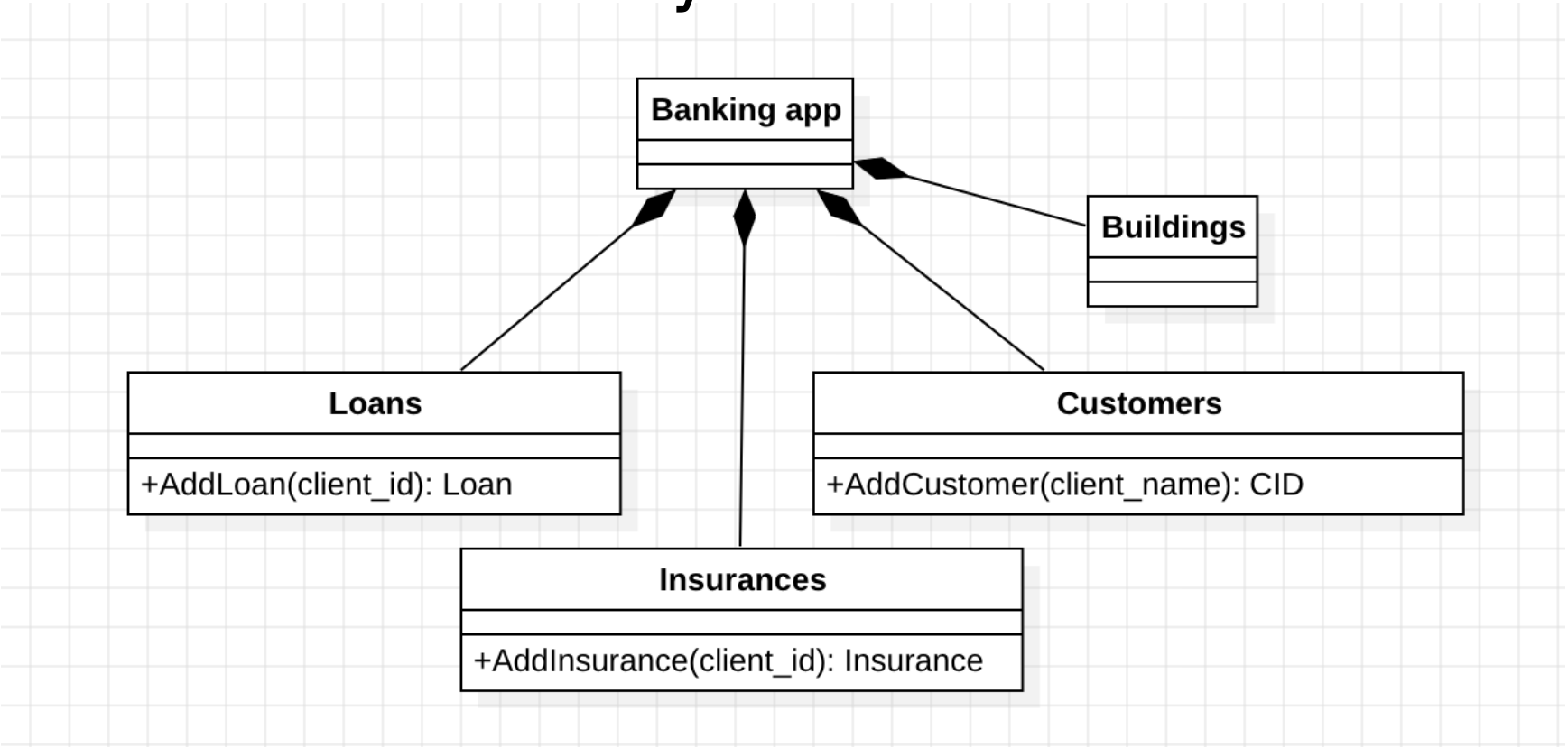
Design patterns need to be put in at the start

- But what if you have an existing app?
  - You did not design in the correct patterns
  - It just grew
    - Organically like, -ish, sort of

Example: Capislack bank

Banking app

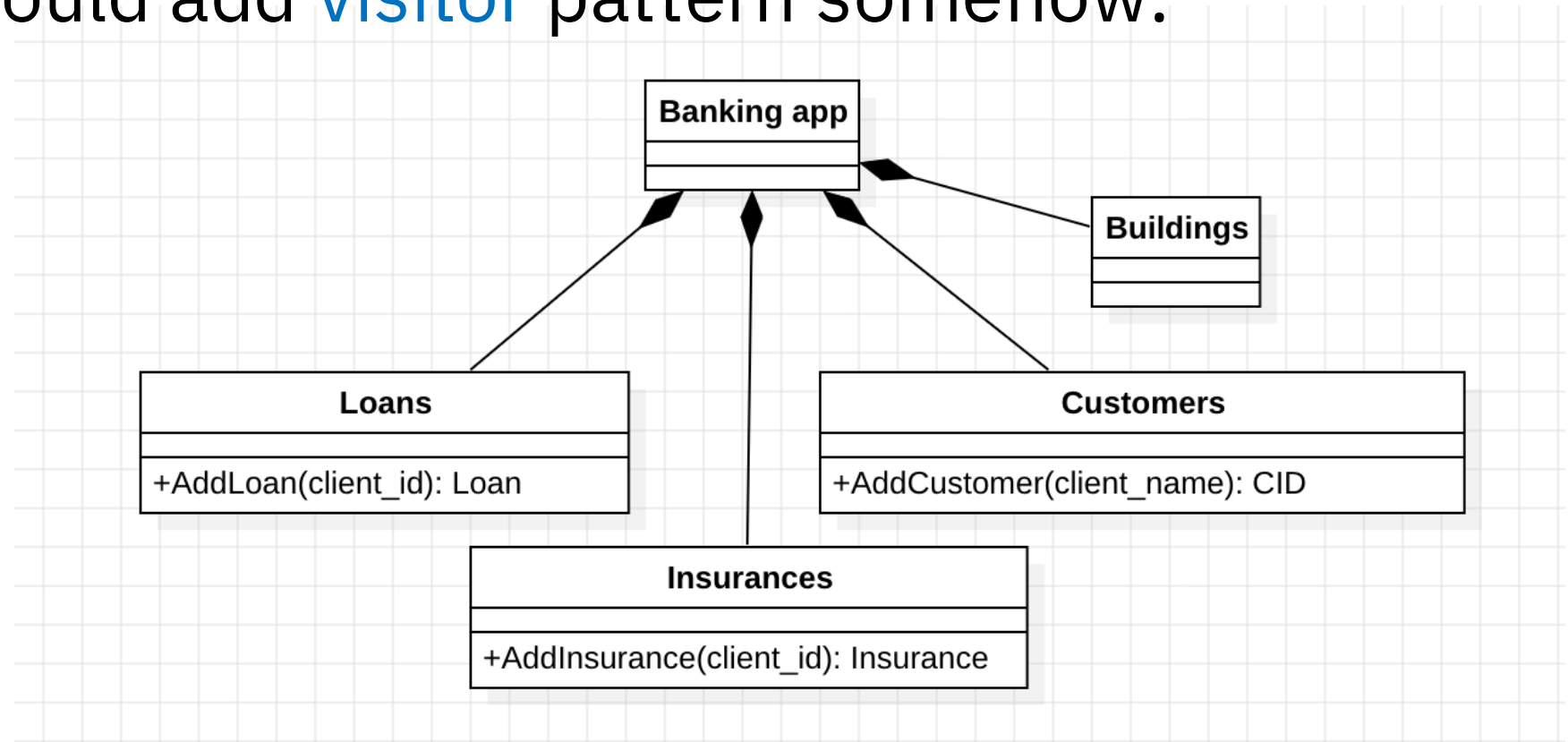
- Grew organically, over 40 years
- Now we need to add in security



Example: Capislack bank

Banking app

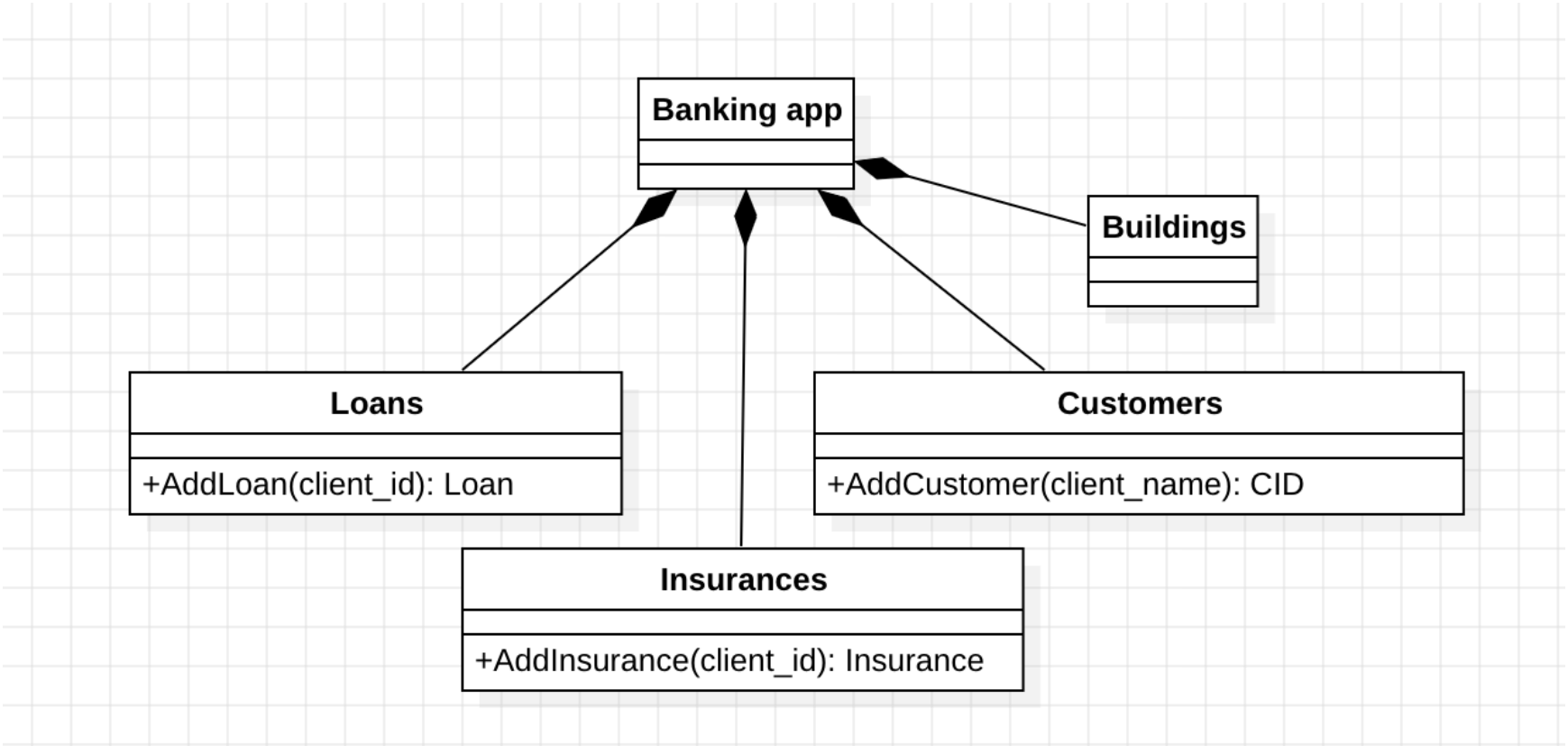
- Adding new features requires adding new operations
- Adding ops in existing OOP app is hard
  - If only we could add visitor pattern somehow.



Aspect oriented programming

Crosscutting concern examples:

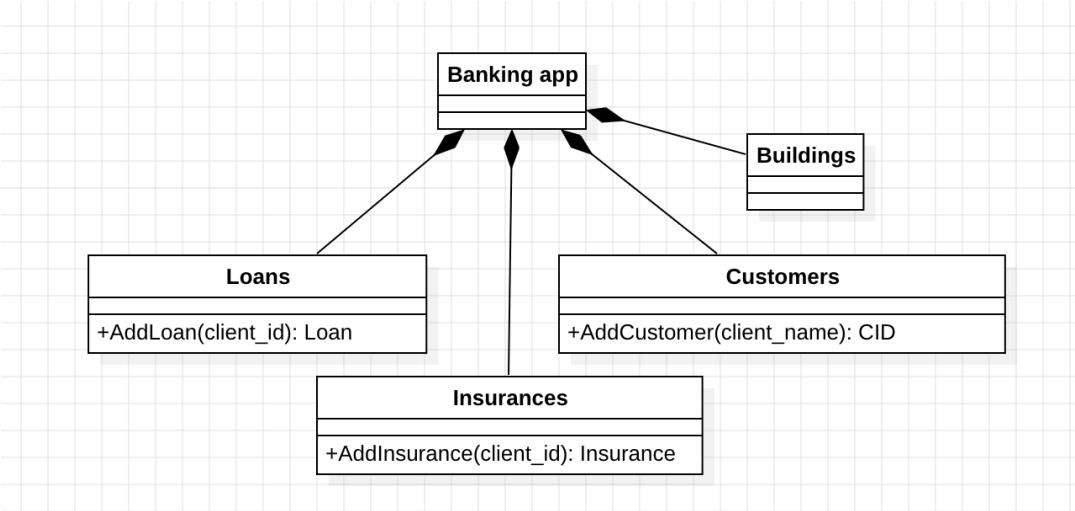
- new transactions only with approved clients
- new clients only if IDs proper & not banned



Aspect oriented programming

Crosscutting concerns:

- Different classes/interfaces need the same *intervention*
- Scattered methods have the same *requirements*
- Same concerns in different parts of application
  - cut across multiple different classes



Advice/Pointcut

Crosscutting concerns are fixed using an advice:

- **Advice**: a piece of code that is *injected* into 1 or more methods of your app.
- **Pointcut**: the location(s) that an advice is injected into
  - before method starts
  - after method ends
  - around the method
- **Aspect**: advice + pointcut

Pointcuts…

Pointcuts are defined using a regular expression

- Match on classname + function signature
  - e.g.: [A-Z]*s.Add*(client_id, *)
    - Match on class starting with capital, ends in 's' with a method that starts with 'Add' and has as its first parameter 'client_id'.
  - e.g. [A-Z]*s.Add*(client_id,*) | *.New*(*name, *)
    - Same, but also match any method starting with 'New' first parameter ends in 'name'
  - Matches can be and'ed (*must match all*) or or'ed (*can match any*) together

Advice

Advice code can run

- before: run prior starting method
- after: run after method has finished, ignore exceptions
- around: run part before, run method, then part after
- after exception: run only when exception happens
- after return: run after if no exceptions

Aspect = Pointcut + Advice

```
@Aspect
public class CheckNewTransactions {
    private Security sec;

    public CheckNewTransactions(Security sec){ this.sec = sec;}

    @Before("execution(* [A-Z]*s.Add*(client_id,..))")
    public void beforeMethodExecution(JoinPoint joinPoint) {
        Object[] args = joinPoint.getArgs();
        if (args[0] instanceof Integer) {
            final var client_id = (int)args[0];
            sec.check(is_customer(client_id));
            sec.check(!is_criminal(client_id));
            sec.check(!is_bankrupt(client_id));
```

How does it work?

Reflection: the ability of a process to examine, introspect, and **modify** its own structure and behaviour

- Dynamic reflection: at runtime
  - Uses RTTI: runtime type information
  - Aka: Injection
- Static reflection: at compile time
  - *The compiler does 'search + replace' on source code*
  - Aka: instrumentation

AspectJ

Java AOP

- Since 2001
- Instruments source code using an extra compile step
- Using the AspectJ *weaving* compiler

Pros and cons

Pro	Con
Modularity: Can easily add crosscutting operations	**Debugging**: class methods only contain part of the code, crosscuts can happen at any time
Code reusability: Makes DRY code easy (apply one aspect in many places)	**Complexity**: Easy to weave very complex crosscuts (Adds an extra dimension to your hierarchy)
Maintainability: easy to patch existing code (even if existing design sub-optimal)	**Tooling**: IDE may make crosscuts hard to see
SOLID: reduces dependencies, separates crosscutting concerns from main logic.	**Portability**: AOP not portable across languages Java has the best support, C++ not yet (maybe C++29)
Scalability: OAP based crosscuts are lightweight	
Testability: Testing simplified because DRY'er code	

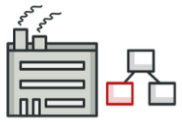
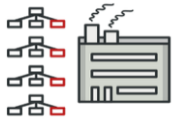
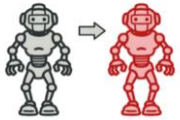
DEMO

Design patterns recap

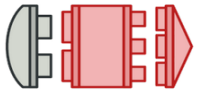
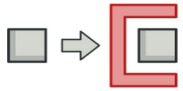
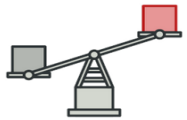
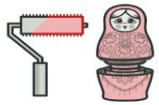
3 Categories

- Creational — Object creation workarounds
- Structural — Mid-level architecture
- Behavioural — Low-level design

Creational patterns recap

	Name	Does what?	Why?
	Factory	Polymorphic construction of single object	No virtual constructors
	Abstract factory	Factory for multiple related objects	" "
	Builder pattern	Construct complex object in parts	Composition
	Singleton	Single instance globally available	Database wrappers/logging
	Prototype	Clone an object (`Object.clone()`)	`a = b` does a shallow copy

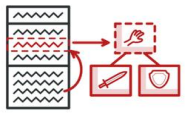
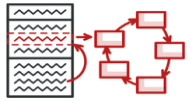
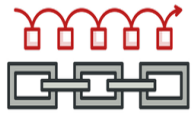
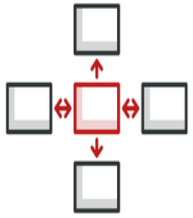
Structural patterns recap

	Name	Does what?	Why?
	Adapter	Connect two mismatching interfaces	Fix mismatching API
	Proxy	Cache slow / encapsulate big object	Speed
	Flyweight	Obsolete	Use DoD instead
	Composite	Recursive container (tree and/or jagged array)	Complex *'list'*
	Facade	Simple interface around complex object	Limit functionality
	Bridge	Prevent combinatorial explosion	10x10=100(bad), 10+10=20(good)
	Decorator	Add new properties	Add colour to shapes

Behavioral patterns - non lambda

	Name	Does what?	Why?
	Memento	Undo / snapshot	Prevent full cloning
	Iterator	For in loop	Magic behind the scenes to make `for in` loop work
	Visitor	Adding behaviour now easy	*Abstract pattern: 3 variants +1 one variant to avoid*
	Cyclic visitor	Easy to add behaviour, hard to add objects	Pure OOP soln
	Acyclic visitor	Easy to add objects + behaviour, but slow…	Dynamic_cast (slow because it walks up and down the inheritance tree)
	Variant visitor Discriminated union/ Tagged union visitor	Easy to add objects + behaviour + fully decoupled 	Variant = Proper 1956 OOP done right
	Procedural visitor	Do not use	Not maintainable

Behavioral patterns – Lambda FtW

	Name	Does what?	Why?/What?
	Command	File -> save (menu + button + <ctrl+s>)	Link lambda to action
	Strategy	Change behaviour of (sort) algorithm	Lambda steers algo
	State	Finite state machine	Use Java enum class instead
	Template	Multi-step algorithm	Lambda per step
	Observer	Subscribe to events	IState interface notifies subscribers of changes Lambda per subscription
	Chain of responsibility	Pipe filter / message passing	MPI: message passing interface (1970's OOP paradigm) Lambda per message handler
	Mediator	Central node of communication	$\frac{n^2-n}{2}$ links => n+1 links Lambda per comm. node Also: MPI